1. Kết quả Benchmarking Off-chain phi thực tế (End-to-End latency)

Trong Section VII, nhóm tác giả khẳng định rằng mạch đệ quy Λ_{agg} (dựa trên Groth16) có khoảng 20 triệu constraints, nhưng lại cho rằng thời gian tạo Proof (Proving time) chỉ tốn 9.840 ± 410 milliseconds trên một CPU 16-thread thông thường.
Dưới góc độ chuyên môn về Mật mã học (Cryptography), đây là một con số phi logic. Việc tính toán Multi-Scalar Multiplications (MSM) và Fast Fourier Transforms (FFT) cho 20 triệu constraints mô phỏng BN254 non-native arithmetic thường đòi hỏi từ 100 đến 300 giây trên các server Cloud chuyên dụng rất mạnh. Rất có thể script benchmark của các em bị lỗi (chỉ chạy compile circuit chứ chưa sinh witness/proof hoàn chỉnh), hoặc các em đã dùng phần cứng tăng tốc (FPGA/GPU) mà không báo cáo minh bạch.
Hệ lụy là toàn bộ Bảng X (End-to-End Round Latency) dự phóng tốc độ toàn trình nhanh hơn 3.1x đến 5.5x sẽ bị bác bỏ, kéo theo thiếu tính thuyết phục của mô hình.
Hướng khắc phục tối ưu (Trong phạm vi bài báo): Tuyệt đối không đổi sang hệ thống Proof khác (như Plonky2 hay Halo2) vì điều này sẽ làm thay đổi toàn bộ kiến trúc Smart Contract, làm mất đi ưu thế lớn nhất của bài là phí Gas trên chuỗi siêu rẻ của Groth16 (127 bytes/proof).

- Bước 1: Chạy lại benchmark trung thực cho quá trình sinh Proof của mạch 20M constraints. Hãy ghi nhận con số thực tế (ví dụ: tốn 150 giây hoặc hơn).
- Bước 2: Tính toán lại Bảng X với con số mới. Tổng thời gian E2E Latency theo thời gian thực chắc chắn sẽ cao hơn so với phương pháp tuyến tính của baseline zkCross.
- Bước 3 (Gỡ điểm bằng lập luận): Thêm một đoạn bàn luận (Discussion) sắc bén: Nhấn mạnh rằng H-ZKA chấp nhận đánh đổi thời gian tính toán Off-chain (vốn có thể giải quyết bằng cách nâng cấp phần cứng, ủy quyền tính toán, hoặc xử lý theo lô/bất đồng bộ) để lấy sự tối ưu tuyệt đối về chi phí On-chain (giảm 13.8 lần phí Gas trên Ethereum). Trong môi trường Blockchain thực tế, rào cản thương mại lớn nhất là phí Gas đắt đỏ chứ không phải độ trễ vài phút ở Off-chain.
- Tác giả cần điều chỉnh các tuyên bố trong phần Abstract và Conclusion, làm rõ rằng H-ZKA không phải là giải pháp vạn năng cho mọi mạng lưới, mà là kiến trúc đặc thù chỉ phát huy hiệu quả tối đa trên các mạng lưới đa chuỗi có mật độ cao (k từ 50 trở lên).
- Bổ sung một tiểu mục thảo luận về kiến trúc phần cứng thực tế, đưa ra giải pháp dùng cơ chế “proof backlog” (xử lý hàng đợi chứng minh) kết hợp với các máy chủ tạo chứng minh song song (parallel provers) để giải thích rõ cách framework này có thể tồn tại trên các chuỗi có thời gian sinh block dưới 12 giây. Việc trung thực về các giới hạn này sẽ biến nhược điểm thành một phân tích thực tiễn xuất sắc, phù hợp với tiêu chuẩn khắt khe của các tạp chí Q1.

2. “Slow-Loris” có công thức toán nhưng không thực nghiệm

Ở Section IV, bài báo đã chỉ ra một kịch bản tấn công rất tinh vi: Kẻ tấn công “Slow-Loris” (Oscillating Attack định kỳ) sẽ gửi 1 proof sai, sau đó “ngoan ngoãn” gửi một số lượng proof đúng nhất định để hệ thống reset bộ đếm phạt, giúp chúng luôn giữ điểm danh tiếng (Reputation) trên mức an toàn ($R_{min} = 0.01$).
Mình đã làm rất tốt về mặt lý thuyết khi đưa ra Theorem 4 (Phương trình 13) với bộ đếm lỗi tích lũy (V_i) không bao giờ reset để chặn đứng lỗ hổng này. Tuy nhiên, ở phần

Hạn chế (Limitations - Section VIII), các em lại thật thà thừa nhận rằng công thức này chưa được tích hợp vào Smart Contract và chưa được chạy mô phỏng.
Trình bày một bản vá bảo mật Toán học quan trọng nhưng lại bỏ ngỏ việc chứng minh tính khả thi của nó trong phần Evaluation là một “red flag”. Các Reviewer sẽ đánh giá bài báo là chưa hoàn thiện (incomplete framework).

Vấn đề này giải quyết rất dễ dàng, chỉ tốn vài giờ code và chạy lại mô phỏng mà không làm thay đổi kiến trúc tổng thể. Phương trình 13 chỉ yêu cầu thêm một biến trạng thái 32-bit (counter) cho mỗi committer trên smart contract Solidity (ReputationRegistry) và chỉnh sửa lại công thức phạt. Nhóm chỉ cần tốn vài giờ để code thêm vài dòng Solidity này, sau đó chạy thêm một kịch bản mô phỏng duy nhất (ví dụ 200 rounds với kẻ tấn công thiết lập chu kỳ N=5) để vẽ ra một biểu đồ chứng minh rằng kẻ tấn công slow-loris rốt cuộc cũng bị đưa về R_min=0.01. Cập nhật biểu đồ này vào bài và xóa bỏ hoàn toàn điểm yếu số (6) trong phần Limitations.

- Bước 1: Mở mã nguồn của Solidity contract (ReputationRegistry), thêm một biến trạng thái uint32 cumulativeSafetyFaults cho mỗi committer.
- Bước 2: Cập nhật hàm tính hệ số phạt β_i^t đúng theo công thức của Phương trình 13. Chi phí Gas tăng thêm cho một phép cộng và cập nhật state này là cực kỳ nhỏ, hoàn toàn không làm ảnh hưởng đến tuyên bố tiết kiệm gas của hệ thống.
- Bước 3: Chạy thêm duy nhất một kịch bản thực nghiệm (tương tự như Figure 3) nhưng áp dụng cho kẻ tấn công Slow-Loris. Vẽ một đồ thị mới (ví dụ: Figure 3b) chứng minh rằng nhờ có biến tích lũy V_i, điểm uy tín của kẻ tấn công theo thời gian vẫn bị hệ thống dìm xuống mức cách ly R_{min}. Cuối cùng, xóa bỏ dòng “chưa implement” trong mục Limitations.